

Boot-Chain Debugging for VMs and Images

Bootloader to application readiness | MTTR 40% lower

The project explained end to end, in simple terms

The project (as written on the resume): Improved MTTR 40% and cut VM and image diagnosis from hours to tens of minutes as measured by incident timelines by debugging guest startup from bootloader through kernel init, initramfs, systemd, and application readiness with QEMU serial consoles, journalctl, dmesg, systemd-analyze, strace, and perf.

What is inside this document

- Section 1 - The simple summary (the big picture)
- Section 2 - Key words explained in plain English
- Section 3 - Architecture diagram
- Section 4 - The architecture diagram explained, piece by piece
- Section 5 - Every component explained in detail
- Section 6 - The complete workflow, step by step
- Section 7 - Error message cheat sheet (symptom to cause)
- Section 8 - How MTTR and diagnosis time were measured
- Section 9 - What changed to make it faster, and a 60-second explanation

Note: The resume line gives the result and the tools. This document explains how this kind of debugging is typically done on Linux VMs (for example on Oracle Cloud and under QEMU/KVM). Exact runbooks and numbers will differ in your real setup. The time numbers in the chart are illustrative examples; your real numbers come from your incident timelines.

1. The Big Picture (Simple Summary)

1.1 The one-line version

When a virtual machine would not start properly, I found the problem much faster by checking its startup one stage at a time - from the very first boot program up to the application being ready - using the right Linux tool for each stage. Diagnosis went from hours to tens of minutes, and the average time to fully fix an incident (MTTR) dropped by 40%.

1.2 Explained with an everyday example

Think of starting a car in the morning. Many things happen in order:

- **1. The key turns and the battery wakes the car** (firmware - BIOS/UEFI).
- **2. The starter motor turns the engine** (bootloader - GRUB).
- **3. The engine catches and runs** (kernel starts).
- **4. Fuel pump and sensors come online** (initramfs finds and opens the main disk).
- **5. Dashboard, lights, radio, and air-con switch on** (systemd starts all services).
- **6. You drive away** (the application is ready and serving users).

If the car does not go, a poor mechanic guesses and tries random fixes for hours. A good mechanic asks one question first: **"What was the last step that worked?"** If the dashboard lights came on but the engine did not catch, the problem is between steps 2 and 3 - so they check only that part, with the right tool.

This project brought that "good mechanic" method to VM and image problems: a clear map of the boot stages, the right tool for each stage, a standard way to collect evidence, and a runbook everyone follows.

1.3 The resume line, phrase by phrase

Phrase	What it means in simple words
Improved MTTR 40%	The average time from "problem detected" to "problem fixed" became 40% shorter. Example: 400 minutes became 240.
cut VM and image diagnosis from hours to tens of minutes	Finding out WHY a VM or image was broken used to take hours; now it takes about 10-40 minutes.
as measured by incident timelines	Each incident records time stamps (detected, diagnosed, fixed). The numbers come from those records.
debugging guest startup	Investigating how a virtual machine (the "guest") starts up.
from bootloader through kernel init, initramfs, systemd	Following every stage of startup in order, from the first boot program to the service manager.

Phrase	What it means in simple words
and application readiness	Up to the moment the actual application is working and answering requests.
QEMU serial consoles	A text window showing every boot message, even when the network is down.
journalctl, dmesg	Tools to read the system logs and the kernel's messages.
systemd-analyze	A tool that shows how long each part of the boot took, and what was slow.
strace, perf	Tools that watch a running program closely: what it asks the system to do (strace) and where it spends CPU time (perf).

1.4 Why this matters

- **Every minute of an incident** can mean customers without their servers.
- **Boot problems are hard:** when a VM does not start, you usually cannot log in to it, so you need tools that work from the outside.
- **Guessing wastes time:** a stage-by-stage method goes straight to the broken part.
- **Same method for everyone:** new team members can solve problems that used to need an expert.
- **Lessons are kept:** every incident improves the runbook and adds tests, so the same problem is caught earlier next time.

2. Key Words in Plain English

Word	Simple meaning	Example
VM / guest	A virtual machine - a computer running inside another computer.	A customer's Linux VM.
Host	The physical machine running the VMs.	A KVM server.
Incident	An unplanned problem that affects a service.	VMs from a new image do not boot.
MTTR	Mean Time To Resolve (or Recover): average time from detection to fix.	240 minutes
Diagnosis time	Time spent finding the cause, not fixing it.	30 minutes
Incident timeline	The list of time-stamped events in an incident.	10:02 detected, 10:40 diagnosed
Boot / startup	Everything that happens from power-on to a working system.	
Firmware (BIOS / UEFI)	The first code that runs; it finds the boot disk. UEFI is the modern type.	OVMF in QEMU
Bootloader (GRUB2)	Program that loads the kernel and initramfs and passes settings to the kernel.	grub.cfg
Kernel command line	Settings given to the kernel at boot.	console=ttyS0 root=UUID=...
Kernel	The core of Linux that talks to hardware.	vmlinuz-5.15...
Kernel init	The kernel's own startup: CPUs, memory, drivers, then it starts the first program.	
initramfs	A tiny temporary system in memory with just enough drivers to find and open the real disk.	initramfs-5.15.img
dracut	Tool that builds the initramfs; also the framework running inside it.	rd.break
switch_root	The moment the system moves from the tiny initramfs to the real disk.	
systemd / PID 1	The first real program; it starts and manages all services.	systemctl
Unit / target	A unit is a service or task; a target is a group of units (a milestone).	sshd.service, multi-user.target
Application readiness	The app is running AND able to serve (port open, health check OK).	HTTP 200 on /health

Word	Simple meaning	Example
Serial console	A text-only screen connected like an old serial cable. Works even if network and graphics are dead.	ttyS0
QEMU	Software that runs virtual machines; can save the serial console to a file.	-serial file:serial.log
dmesg	Shows the kernel's message buffer.	dmesg -T
journalctl	Reads the systemd journal - logs from the kernel, services, and boot.	journalctl -b -p err
Persistent journal	Logs saved to disk so they survive a reboot.	/var/log/journal
systemd-analyze	Shows boot timing and the slow chain of services.	systemd-analyze blame
strace	Shows every request (system call) a program makes to the kernel.	strace -p 1234
System call	A program asking the kernel to do something: open a file, connect, read.	open(), connect()
perf	Linux performance tool; shows where CPU time is spent.	perf top
Emergency shell	A basic rescue prompt when boot cannot continue.	dracut:/#
Kernel panic	A fatal kernel error; the system stops.	"Kernel panic - not syncing"
Runbook	A written step-by-step guide for handling a type of problem.	"VM won't boot" runbook
Postmortem	A blameless write-up after an incident: what happened and how to prevent it.	

3. Architecture Diagram

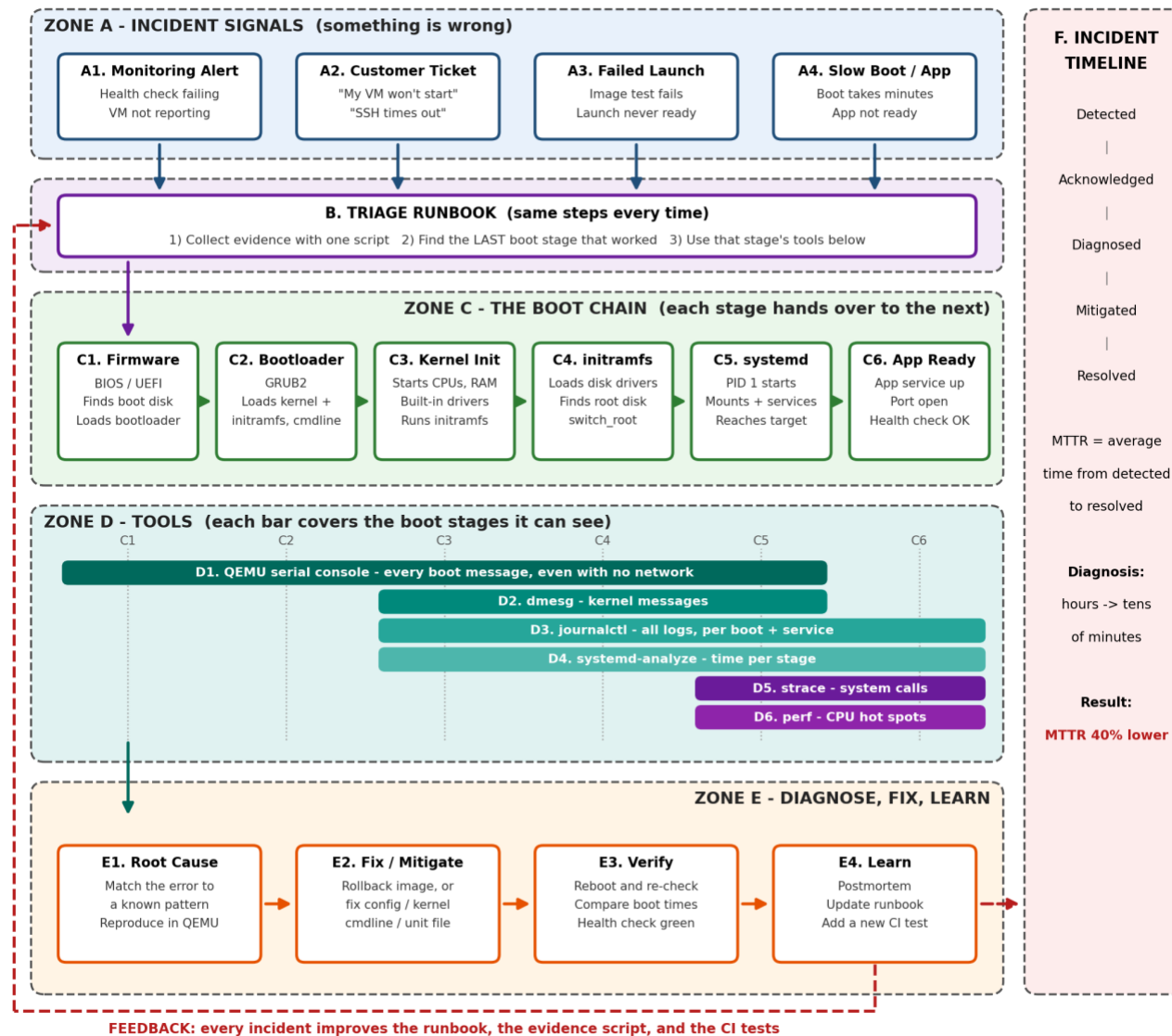


Figure 1 - How incidents flow through triage, the boot chain, the tools, and the fix. Labels (A1-F) match Sections 4 and 5.

4. The Architecture Diagram Explained, Piece by Piece

4.1 How to read the diagram

Visual element	What it means
Big dashed boxes (zones A-F)	Groups of parts with the same job: signals, triage, the boot chain, tools, fixing, and measuring.
Small solid boxes	Individual components. Labels match Section 5.
Blue (Zone A)	How we learn something is wrong.
Purple (Zone B)	The triage runbook - the standard first steps.
Green (Zone C)	The six stages of VM startup, in order.
Teal and purple bars (Zone D)	The tools. Each bar is stretched across the boot stages that tool can see. Grey dotted lines and the labels C1-C6 line the bars up with the stages above.
Orange (Zone E)	Finding the root cause, fixing, verifying, and learning.
Red side bar (Zone F)	The incident timeline and the measured results.
Solid arrows	The order of work.
Red dashed arrows	Results going into measurement, and the feedback loop back to the runbook.

4.2 Zone A - Incident signals

- **A1 Monitoring Alert:** automated health checks fail or the VM stops reporting.
- **A2 Customer Ticket:** a customer reports that a VM will not start or cannot be reached.
- **A3 Failed Launch:** an image test or a new launch never becomes ready.
- **A4 Slow Boot / App:** the VM starts but takes too long, or the app is not ready.
- **Blue arrows:** every signal goes into the same triage runbook.

4.3 Zone B - Triage runbook

One wide box because it is the single front door. It has three steps: **collect evidence** with one script, **find the last boot stage that worked**, and **use that stage's tools**. The **purple arrow** leads into the boot chain.

4.4 Zone C - The boot chain

- **C1 Firmware:** BIOS or UEFI finds the boot disk and loads the bootloader.

- **C2 Bootloader:** GRUB2 loads the kernel and initramfs and passes the kernel command line.
- **C3 Kernel Init:** the kernel starts CPUs and memory, sets up built-in drivers, and runs the initramfs.
- **C4 initramfs:** loads disk drivers, finds the root disk, and switches to it.
- **C5 systemd:** starts as PID 1, mounts filesystems, starts services, and reaches its target.
- **C6 App Ready:** the application service is running, its port is open, and its health check passes.
- **Green arrows:** each stage hands control to the next. A failure at any stage stops everything after it - which is why finding the last good stage is so powerful.

4.5 Zone D - Tools and what they can see

Bar	Covers stages	Why it covers those stages
D1 QEMU serial console	C1 to C5	Shows everything printed to the console - firmware, GRUB, kernel, initramfs, and systemd messages - even when the VM has no network. Works from outside the VM.
D2 dmesg	C3 to C5	The kernel's message buffer starts when the kernel starts and keeps running.
D3 journalctl	C3 to C6	The journal collects kernel messages, initramfs messages, and all service and app logs.
D4 systemd-analyze	C3 to C6	Reports time for kernel, initramfs, and userspace (and firmware/loader on UEFI), plus which services were slow.
D5 strace	C5 to C6	Attaches to a running service or app to see exactly what it is waiting for.
D6 perf	C5 to C6	Shows where CPU time goes in services and apps, for slow starts.

Teal arrow from Zone D down to E1: the evidence from the tools leads to the root cause.

4.6 Zone E - Diagnose, fix, learn

- **E1 Root Cause:** match the error to a known pattern and reproduce it in local QEMU.
- **E2 Fix / Mitigate:** roll back to the last good image first if customers are affected, then fix the real cause (config, kernel, command line, unit file).
- **E3 Verify:** reboot, re-check the same evidence, compare boot times, confirm health checks are green.
- **E4 Learn:** postmortem, update the runbook, and add a CI test so it is caught before release next time.
- **Orange arrows:** these steps happen in order.

4.7 Zone F and the feedback loop

Zone F lists the incident timeline milestones: detected, acknowledged, diagnosed, mitigated, resolved. MTTR is the average time from detected to resolved. The **red dashed arrow** from E4 sends each incident's times here. The **red dashed line** around the bottom and left side goes from E4 back into the triage runbook (B): every incident improves the runbook, the evidence script, and the tests.

4.8 Every arrow in one table

From	To	What travels along the arrow
A1-A4	B	The signal that something is wrong.
B	C1	Evidence collected; start checking stages in order.
C1 to C6	Next stage	Control of the boot process (and where to look next).
D	E1	Log lines, timings, and traces that point to the cause.
E1 to E4	Next step	Cause, then fix, then proof, then lessons.
E4	F	Incident time stamps for MTTR measurement (dashed).
E4	B	Runbook and tool improvements (feedback loop, dashed).

5. Every Component Explained in Detail

Each component has four parts: **In simple terms**, **Detailed breakdown**, **Why it matters**, and an **Everyday analogy**. Many also show example commands (grey boxes). Labels match the diagram.

A1 - Monitoring Alert

In simple terms

An automatic alarm that something is unhealthy.

Detailed breakdown

- **Health checks** (ping, SSH, HTTP) fail.
- **Agent silence**: the VM's monitoring agent stops sending data.
- **Launch success rate** drops for a new image.
- **Alert opens an incident** and records the first time stamp: detected.

Why it matters

The earlier an incident is detected, the shorter the MTTR.

Everyday analogy: A warning light on the dashboard.

A2 - Customer Ticket

In simple terms

A customer reports a problem.

Detailed breakdown

- **Typical words**: "VM stuck starting", "cannot SSH", "app not responding after reboot".
- **Key facts collected**: instance ID, image version, shape, time, what changed.
- **Linked** to the incident so the timeline is complete.

Why it matters

Real user impact; often the first sign of a problem monitoring missed.

Everyday analogy: A customer calling the garage.

A3 - Failed Launch

In simple terms

A new VM or image test never becomes ready.

Detailed breakdown

- **CI boot test** times out.
- **New VM** stays in a starting state or fails its health check.
- **Often image-related**: a new kernel, driver, or config.

Why it matters

Catching image problems here prevents wider incidents.

Everyday analogy: A new car that fails its first test drive.

A4 - Slow Boot / App Not Ready

In simple terms

The VM starts, but too slowly, or the app never becomes ready.

Detailed breakdown

- **Boot time** much longer than normal.
- **App stuck** "starting" - waiting for something.
- **Needs timing tools** (systemd-analyze, strace, perf) rather than error logs.

Why it matters

Slow is also broken - it delays scaling and recovery.

Everyday analogy: The car starts, but takes five minutes to warm up.

B - Triage Runbook (the front door)

In simple terms

The same three first steps for every VM or image problem.

Detailed breakdown

- **Step 1 - collect evidence** with one script, before changing anything: serial console log, journal (current and previous boot), dmesg, systemd-analyze outputs, failed units, image version, kernel command line.
- **Step 2 - find the last stage that worked** by reading the serial log from the top: did GRUB appear? did the kernel print messages? did the initramfs switch root? did systemd reach its target?
- **Step 3 - go to that stage's section** of the runbook with the right tools and known error patterns.
- **Mitigate early:** if customers are affected and an image change is suspected, roll back first, then diagnose.
- **Record time stamps** (acknowledged, diagnosed) in the incident.

Example

```
# evidence script (simplified)
journalctl -b --no-pager > journal.txt
journalctl -b -1 --no-pager > journal-prev.txt
dmesg -T > dmesg.txt
systemd-analyze time > sa-time.txt; systemd-analyze blame > sa-blame.txt
systemctl --failed > failed.txt; cat /proc/cmdline > cmdline.txt
```

Why it matters

Removes guessing. Everyone starts the same way and goes straight to the broken stage.

Everyday analogy: A doctor's standard intake: temperature, pulse, blood pressure, then focused tests.

C1 - Firmware (BIOS / UEFI)

In simple terms

The first code that runs. It checks basic hardware and finds something to boot.

Detailed breakdown

- **What happens:** virtual hardware check, find the boot disk, load the bootloader (BIOS reads the start of the disk; UEFI reads the EFI System Partition).
- **Good sign:** firmware messages then GRUB appears on the serial console.
- **Common failures:** "No bootable device", wrong firmware type for the image (BIOS image on UEFI or the reverse), missing EFI partition, Secure Boot rejecting unsigned files.
- **How to see it:** QEMU serial console; for UEFI, OVMF debug output can be enabled in QEMU.

Why it matters

If this stage fails, nothing inside the VM runs - no logs exist inside the VM at all. Only outside tools help.

Everyday analogy: Turning the key: does the battery wake the car?

C2 - Bootloader (GRUB2)

In simple terms

Loads the kernel and initramfs and tells the kernel its settings.

Detailed breakdown

- **What happens:** read grub.cfg, show the menu, load the kernel (vmlinuz) and initramfs into memory, pass the kernel command line.
- **Good sign:** GRUB menu then "Loading Linux..." on the serial console (GRUB must be configured to use the serial terminal).
- **Common failures:** "grub rescue>" prompt, "file not found", wrong root UUID, missing kernel after an update, missing console=ttyS0 (boot continues but you see nothing).
- **How to debug:** serial console; edit the entry at the GRUB menu to add debug options for one boot.

Example

```
# GRUB settings so boot is visible on serial
GRUB_TERMINAL="console serial"
GRUB_SERIAL_COMMAND="serial --speed=115200"
GRUB_CMDLINE_LINUX="console=tty0 console=ttyS0,115200"
```

Why it matters

Controls which kernel starts and with which settings - a single wrong setting breaks the boot.

Everyday analogy: The starter motor: turns the engine and sets the choke.

C3 - Kernel Init

In simple terms

The kernel starts itself: CPUs, memory, built-in drivers, then runs the initramfs.

Detailed breakdown

- **What happens:** decompress, detect CPUs and memory, set up interrupts and built-in drivers, unpack the initramfs into memory, run its /init program.
- **Good sign:** a stream of kernel messages with time stamps, then initramfs messages.
- **Common failures:** kernel panic, "VFS: Unable to mount root fs", hang with no messages, oops or call trace, CPU feature problems on some machine types.
- **How to debug:** serial console with console=ttyS0; more detail with loglevel=7 or ignore_loglevel; very early problems with earlycon/earlyprintk; after boot, dmesg and journalctl -k.

Why it matters

Kernel problems affect every VM using that image and are the most serious.

Everyday analogy: The engine catching and running.

C4 - initramfs

In simple terms

A tiny temporary system that loads disk drivers, finds the real root disk, and hands over to it.

Detailed breakdown

- **What happens:** load drivers (virtio, NVMe, iSCSI), wait for the root disk to appear by UUID, optional steps (LVM, fsck, network disk login), mount the root at /sysroot, switch_root to it.
- **Good sign:** messages about reaching the initrd targets and switching root.
- **Common failures:** "dracut-initqueue timeout", "/dev/disk/by-uuid/... does not exist", drop to the dracut emergency shell, wrong UUID after an image change, missing driver after a kernel update.
- **How to debug:** serial console; rd.debug for detailed output; rd.break to stop just before switching root and inspect; journalctl -b later shows initramfs logs.

Example

```
# one-time kernel options for debugging (add at GRUB menu)
rd.debug          # detailed initramfs log
rd.break          # stop before switch_root, get a shell
```

Why it matters

The most common reason images fail to boot after kernel or driver changes.

Everyday analogy: The fuel pump starting so the engine gets fuel from the main tank.

C5 - systemd

In simple terms

The first real program (PID 1). It mounts filesystems and starts all services in the right order.

Detailed breakdown

- **What happens:** read unit files, mount /etc/fstab entries, start services in parallel respecting dependencies, reach targets (sysinit, basic, multi-user).
- **Good sign:** systemctl is-system-running says "running"; login prompt on serial console.
- **Common failures:** "A start job is running for..." (waiting for a disk or network), emergency mode because of a bad fstab line, failed units, dependency cycles, cloud-init or network service failing.
- **How to debug:** systemctl --failed, systemctl status name, journalctl -b -u name, systemctl list-jobs (what boot is waiting on), systemd-analyze blame and critical-chain; systemd.log_level=debug for detail.

Why it matters

Most "VM boots but SSH does not work" problems live here.

Everyday analogy: Dashboard, lights, radio, and air-con all switching on.

C6 - Application Readiness

In simple terms

The final stage: the application is not just started but able to serve.

Detailed breakdown

- **What happens:** the app service starts, loads config, connects to databases or other services, opens its port, passes its health check.
- **Good sign:** port listening (ss -ltnp) and health endpoint returns OK.
- **Common failures:** app waits forever on DNS or a network connection, permission denied, missing file or secret, very slow start because of heavy CPU work.
- **How to debug:** journalctl -u app, systemd-analyze critical-chain app.service, strace to see what it waits for, perf to see where CPU goes.

Why it matters

Customers care about the app, not the boot. "Booted" is not the same as "ready".

Everyday analogy: Actually driving away and arriving.

D1 - QEMU Serial Console

In simple terms

A text-only screen that records every boot message from the outside of the VM.

Detailed breakdown

- **Works with no network**, no SSH, and no graphics - the only view into early boot.
- **QEMU** can save it to a file (-serial file:serial.log) or show it live (-nographic).
- **In the cloud**, the same output is available through the provider's serial console connection (for example OCI console connection).
- **Needs the image to be ready**: GRUB serial terminal and console=ttyS0 on the kernel command line.
- **Read from the top**: the last clean message shows the last stage that worked.
- **Interactive use**: reach the GRUB menu, add debug options, or use an emergency shell.

Example

```
qemu-system-x86_64 -enable-kvm -m 2048 -nographic \
-drive file=vm.qcow2,if=virtio -serial file:serial.log
```

Why it matters

The single most important tool for "VM won't boot" problems.

Everyday analogy: A flight recorder for the boot.

D2 - dmesg

In simple terms

Reads the kernel's own message buffer.

Detailed breakdown

- **Shows**: hardware detection, driver loading, disk and network device errors, oops and call traces, out-of-memory kills.
- **Useful options**: -T for human-readable times, --level=err,warn to filter, -w to watch live.
- **Limit**: the buffer is in memory and lost on reboot (use journalctl -k -b -1 with a persistent journal for the previous boot).

Example

```
dmesg -T --level=emerg,alert,crit,err,warn
dmesg -T | grep -iE 'virtio|nvme|i/o error|oom|call trace'
```

Why it matters

Kernel and driver problems show up here first.

Everyday analogy: The engine's own diagnostic codes.

D3 - journalctl

In simple terms

Reads the systemd journal - one place for kernel, boot, service, and app logs.

Detailed breakdown

- **-b** current boot; **-b -1** previous boot (needs a persistent journal - the images were set up with one so the failed boot's logs survive); **--list-boots** to see all boots.
- **-p err** only errors; **-u name** one service; **-k** only kernel.
- **-o short-monotonic** shows seconds since boot, great for seeing where time went.
- **--since / --until** to focus on the incident window.

Example

```
journalctl --list-boots
journalctl -b -1 -p err --no-pager
journalctl -b -u sshd -o short-monotonic
```

Why it matters

The main evidence source once the system got past the kernel.

Everyday analogy: The car's full service logbook.

D4 - systemd-analyze

In simple terms

Shows how long the boot took and which parts were slow.

Detailed breakdown

- **time:** split into kernel, initrd, and userspace (plus firmware and loader on UEFI).
- **blame:** list of services by start-up time.
- **critical-chain:** the chain of services the boot actually waited on - the real bottleneck.
- **critical-chain app.service:** what the app waited for.
- **plot:** a picture (SVG) of the whole boot timeline.
- **verify:** checks unit files for mistakes.

Example

```
systemd-analyze time
systemd-analyze blame | head
systemd-analyze critical-chain
systemd-analyze plot > boot.svg
```

Why it matters

Turns "boot is slow" into "this one service costs 90 seconds".

Everyday analogy: Lap times for each part of a race.

D5 - strace

In simple terms

Watches every request a program makes to the kernel, so you can see exactly what it is doing or waiting for.

Detailed breakdown

- **Attach** to a running process (-p PID) or start one under strace.
- **-f** follow child processes; **-tt -T** time stamps and time spent in each call.
- **-c** summary of which calls take the most time.
- **What to look for:** ENOENT (file not found), EACCES (permission denied), connect() taking seconds (network or DNS wait), a program stuck in one read() or poll().
- **Careful:** slows the program; use briefly.

Example

```
strace -f -tt -T -p $(pidof myapp) -o /tmp/app.trace
strace -c -f -p $(pidof myapp)
```

Why it matters

Answers "why is the app stuck?" when the logs say nothing.

Everyday analogy: Listening to every phone call a person makes to see who they are waiting on.

D6 - perf

In simple terms

Shows where CPU time is being spent, down to the function.

Detailed breakdown

- **perf top:** live view of the hottest functions.
- **perf record -g / perf report:** record a profile with call stacks, then study it.
- **perf stat:** counts of CPU events (cycles, context switches).
- **Use for:** slow start that is CPU-bound (not waiting), kernel hot spots, spin loops.
- **Flame graphs** can be made from perf data to see hot paths at a glance.

Example

```
perf top -p $(pidof myapp)
perf record -g -p $(pidof myapp) -- sleep 30
perf report --stdio | head -50
```

Why it matters

When a program is busy (not stuck), perf shows what it is busy with.

Everyday analogy: A heat camera showing which engine part is running hot.

E1 - Root Cause

In simple terms

Turning evidence into one clear cause.

Detailed breakdown

- **Match the error** to a known pattern (see the cheat sheet in Section 7).
- **Compare** with the last good image or a healthy VM: what changed (kernel, packages, config)?
- **Reproduce** in local QEMU with the same image and settings, where you can reboot and add debug options freely.
- **Record** "diagnosed" time stamp.

Why it matters

Fixing the symptom without the cause brings the incident back.

Everyday analogy: The mechanic naming the broken part.

E2 - Fix / Mitigate

In simple terms

Stop the damage first, then fix the cause.

Detailed breakdown

- **Mitigate:** roll back to the last good image or kernel; boot the previous GRUB entry; pause a rollout.
- **Fix:** correct the config, fstab, unit file, kernel command line, initramfs drivers, or package.
- **Fix goes into the image recipe**, not only on one VM, so new VMs are correct too.
- **Record** "mitigated" and "resolved" time stamps.

Why it matters

Customers recover fast, and the real fix is permanent.

Everyday analogy: Towing the car to safety, then replacing the part.

E3 - Verify

In simple terms

Prove the fix works.

Detailed breakdown

- **Reboot** and read the same evidence again (serial log, failed units, errors).
- **Compare boot time** with systemd-analyze before and after.
- **Health check** green; app ready.
- **Test on several machine types** if the problem was hardware- or firmware-related.

Why it matters

Avoids closing an incident that is not really fixed.

Everyday analogy: A test drive after the repair.

E4 - Learn

In simple terms

Make sure the next incident is shorter or never happens.

Detailed breakdown

- **Blameless postmortem** with the timeline.
- **Runbook update:** add the new error pattern and its fix.
- **Evidence script update** if something was missing.
- **New CI test** so the same problem is caught before release.
- **Image improvements** such as serial console and persistent journal enabled by default.

Why it matters

This is where the long-term MTTR improvement comes from.

Everyday analogy: Updating the repair manual.

F - Incident Timeline and Metrics

In simple terms

The time stamps that prove the improvement.

Detailed breakdown

- **Milestones:** detected, acknowledged, diagnosed, mitigated, resolved.
- **MTTR** = average of (resolved - detected) over all incidents in a period.
- **Diagnosis time** = diagnosed - acknowledged.
- **Compare** similar periods and incident types before and after the new method.
- **Result:** diagnosis from hours to tens of minutes; MTTR 40% lower.

Why it matters

Numbers from real records, not opinions.

Everyday analogy: A stopwatch on every repair job.

6. The Complete Workflow, Step by Step

This follows one incident from the first alert to the lessons learned.

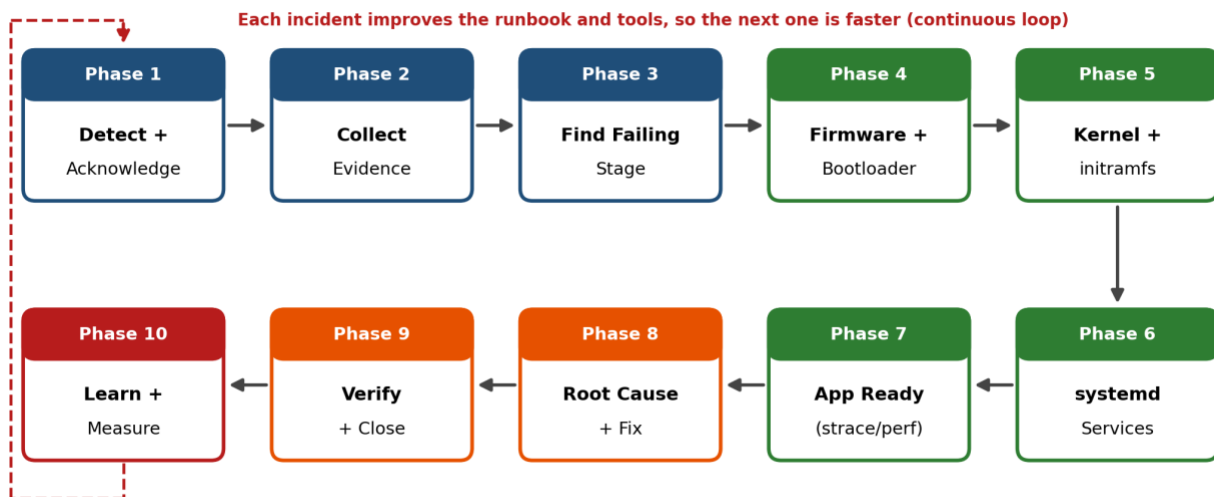


Figure 2 - The 10 phases. Blue = start, green = stage-by-stage debugging, orange = fix and verify, red = learn and measure.

Phase 1 - Detect and acknowledge

Step	What happens (detail)	In simple words
1.1	Alert, ticket, or failed test opens an incident; "detected" time recorded.	The warning light comes on.
1.2	On-call engineer acknowledges; "acknowledged" time recorded.	Someone picks up the job.
1.3	Scope: one VM or many? one image version? one machine type? since when?	How big is it?
1.4	If many customers are affected and a new image is suspected, roll back or pause first.	Stop the damage early.

Phase 2 - Collect evidence

Step	What happens (detail)	In simple words
2.1	Get the serial console log (QEMU file or cloud console connection).	Grab the flight recorder.
2.2	If the VM is reachable, run the evidence script: journal (this and previous boot), dmesg, systemd-analyze, failed units, cmdline.	Take all the readings at once.

Step	What happens (detail)	In simple words
2.3	Note image version, kernel version, machine type, and recent changes.	What is different?

Phase 3 - Find the failing stage

Step	What happens (detail)	In simple words
3.1	Read the serial log from the top and find the last healthy message.	Where did it stop?
3.2	Map it to a stage: firmware, GRUB, kernel, initramfs, systemd, or app.	Which part of the car?
3.3	Open that stage's runbook section and known error patterns.	Open the right chapter of the manual.

Phase 4 - Firmware and bootloader (if stuck there)

Step	What happens (detail)	In simple words
4.1	No bootloader output: check firmware type (BIOS vs UEFI), EFI partition, Secure Boot.	Does the key wake the car?
4.2	GRUB errors: check grub.cfg, kernel files present, root UUID, serial settings.	Is the starter working?
4.3	Boot a previous GRUB entry to confirm a kernel-specific problem.	Try the spare starter.

Phase 5 - Kernel and initramfs (if stuck there)

Step	What happens (detail)	In simple words
5.1	Kernel panic or oops: read the message and call trace; check kernel version change.	Engine fault code.
5.2	Root disk not found: check initramfs has the disk driver and the UUID is right.	Is fuel reaching the engine?
5.3	Reproduce in local QEMU with rd.debug or rd.break; inspect with lsinitrd.	Test it on the bench.

Phase 6 - systemd (if stuck there)

Step	What happens (detail)	In simple words
6.1	systemctl --failed and systemctl list-jobs to see what failed or what boot is waiting for.	Which switch did not turn on?
6.2	journalctl -b -u unit for the failing service; check fstab if in emergency mode.	Read that part's log.
6.3	systemd-analyze blame and critical-chain for slow boots.	Which part is slow?

Phase 7 - Application readiness (if the app is the problem)

Step	What happens (detail)	In simple words
7.1	journalctl -u app and systemd-analyze critical-chain app.service.	What was the app waiting on?
7.2	If stuck: strace -p to see the system call it waits on (file, network, DNS, lock).	Listen to its calls.
7.3	If busy: perf top / perf record to find the hot code.	Use the heat camera.
7.4	Check the port and health endpoint.	Can it actually serve?

Phase 8 - Root cause and fix

Step	What happens (detail)	In simple words
8.1	Name the root cause and record "diagnosed" time.	Name the broken part.
8.2	Mitigate if not already done; apply the real fix in the image recipe or config.	Repair it properly.
8.3	Record "mitigated" and "resolved" times.	Log the repair time.

Phase 9 - Verify and close

Step	What happens (detail)	In simple words
9.1	Reboot; re-collect evidence; no errors or failed units.	Test drive.
9.2	Compare boot time before and after; health checks green.	Check it runs well, not just runs.
9.3	Close the incident with a summary.	Hand the keys back.

Phase 10 - Learn and measure

Step	What happens (detail)	In simple words
10.1	Blameless postmortem with the full timeline.	Review the job.
10.2	Update runbook, error patterns, and evidence script.	Update the manual.
10.3	Add a CI test for the failure.	Catch it at the factory next time.
10.4	Timeline data feeds MTTR and diagnosis-time dashboards.	Update the scoreboard.

7. Error Message Cheat Sheet (Symptom to Cause)

A big part of the speed-up: common messages were mapped to their stage, cause, and tool, so engineers recognize them in seconds.

What you see	Stage	Likely cause	Tool / next step
Nothing at all on the console	C1-C2	Wrong firmware type, missing EFI files, or no serial console configured	Serial console; check BIOS/UEFI setting and GRUB serial config
"No bootable device"	C1	Disk not found or boot partition missing	Firmware messages; check partitions
"grub rescue>"	C2	GRUB cannot find its files or config	Check /boot, grub.cfg, UUIDs
Stops after "Loading initial ramdisk"	C2-C3	Console not set to ttyS0, or very early kernel failure	Add console=ttyS0, earlycon
"Kernel panic - not syncing: VFS: Unable to mount root fs"	C3-C4	Missing initramfs or missing disk driver	lsinitrd; rebuild initramfs with driver
"dracut-initqueue timeout" / "does not exist"	C4	Root disk not found: wrong UUID or driver	rd.debug, rd.break; check root= and blkid
Oops / call trace	C3-C5	Kernel or driver bug	dmesg, journalctl -k; try previous kernel
"A start job is running for ..." (counting up)	C5	Waiting for a disk or network that never appears	systemctl list-jobs; check fstab / network
"You are in emergency mode"	C5	Bad fstab entry or failed mount	journalctl -b -p err; fix fstab (nofail for extra disks)
"Failed to start ...service"	C5	Service config or dependency problem	systemctl status; journalctl -u
Boots, but no SSH	C5	cloud-init, network, or sshd failure	journalctl -u cloud-init -u sshd; ip addr
Boot takes minutes	C5	One slow service on the critical path	systemd-analyze critical-chain / blame
App "starting" forever	C6	Waiting on DNS, network, lock, or file	strace -p (look for connect/poll/ENOENT)
App slow to start, CPU at 100%	C6	Heavy CPU work at start	perf top / perf record
"Out of memory: Killed process"	C5-C6	Not enough memory for services	dmesg; check memory size and app limits

8. How MTTR and Diagnosis Time Were Measured

Illustrative example: diagnosis 180 -> 30 min; total 400 -> 240 min = 40% lower MTTR

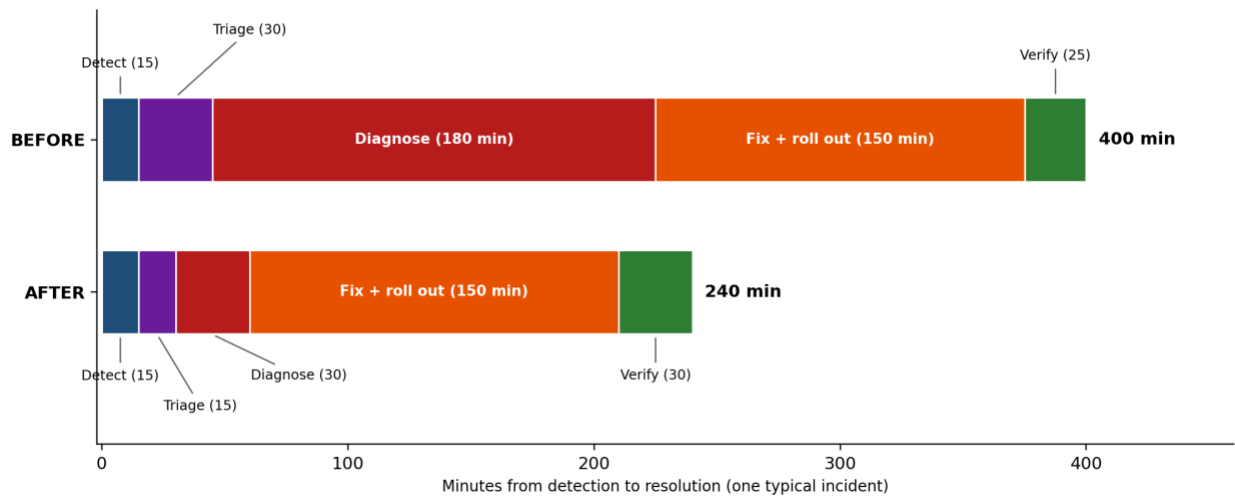


Figure 3 - Illustrative only. The fix and roll-out time stays similar; the big saving is in diagnosis.

- **Every incident has a timeline** with time stamps: detected, acknowledged, diagnosed, mitigated, resolved.
- **MTTR** = average of (resolved - detected) across incidents in a period.
- **Diagnosis time** = diagnosed - acknowledged.
- **Compare like with like:** similar VM and image incidents, before and after the new method (for example two quarters each).
- **Formula:** improvement = (old MTTR - new MTTR) / old MTTR. Example: $(400 - 240) / 400 = 40\%$.
- **Why MTTR fell 40% but diagnosis fell much more:** other parts of an incident (building a fix, approvals, rolling it out, verifying) did not get faster. Diagnosis was the biggest part that changed.

9. What Changed to Make It Faster, and a 60-Second Explanation

9.1 Before vs. after

Area	Before	After
Starting point	Guess and try fixes	Find the last stage that worked
Evidence	Collected by hand, often incomplete	One script collects everything, before changes
Early boot visibility	Often none (no serial output)	Serial console enabled in all images (GRUB + console=ttyS0)
Previous-boot logs	Lost after reboot	Persistent journal (journalctl -b -1)
Slow boots	"It feels slow"	systemd-analyze blame / critical-chain gives the exact service
Stuck apps	Restart and hope	strace shows what it waits on; perf shows CPU hot spots
Reproduction	In production, slowly	Local QEMU with debug options (rd.debug, rd.break, systemd.log_level=debug)
Knowledge	In a few experts' heads	Runbook + error cheat sheet everyone uses
Learning	Same problems repeat	Postmortems add runbook entries and CI tests

9.2 The 60-second explanation (for interviews)

"VM and image incidents used to take hours to diagnose because people guessed. I introduced a stage-based method for the whole startup chain - firmware, GRUB, kernel init, initramfs, systemd, and application readiness. We enabled serial console output and persistent journals in our images and wrote one evidence script. The runbook starts by finding the last stage that worked in the QEMU or cloud serial log, then uses the right tool: dmesg and journalctl for kernel and initramfs issues, systemctl and systemd-analyze blame and critical-chain for service failures and slow boots, and strace and perf for apps that were stuck or slow. We reproduced issues in local QEMU with debug options like rd.debug and rd.break. Incident timelines showed diagnosis dropping from hours to tens of minutes and MTTR improving by 40%, and every postmortem added runbook entries and CI tests."